Deducción natural en lógica proposicional

Tema 1 — Razonamiento Automático

Fabián Fernando Serrano Suárez

Isabelle/HOL

Ciencias de la Computación

Cómo se lee una demostración

Anatomía de una demostración en Isar

```
lemma ejemplo_1_1:
  assumes 1: "p ∧ q" and
          2: "r"
  shows "q ∧ r"
proof -
  have 3: "q" using 1 by (rule conjunct2)
  show 4: "q ∧ r" using 3 2 by (rule conjI)
qed
```

assumes para indicar las hipótesis
and separa las hipótesis
shows la conclusión
proof - prueba sin método por defecto

have un paso intermedio
using qué hechos usa el paso
by (rule R) con qué regla
show / **qed** la conclusión y el cierre.

Cada paso dice explícitamente qué hechos se utilizan y con qué reglas.

Dos niveles de flecha

Símbolo	Se lee	Nivel
$\longrightarrow$	«si ... entonces»	fórmula del objeto
$\Longrightarrow$	«de ... se deduce»	regla del meta-nivel
$[\![A;\, B]\!] \Longrightarrow C$	«de A y B se deduce C»	regla con dos premisas
$\vdash$	«... demuestra ...»	el enunciado del ejercicio

$p \longrightarrow q$ es una fórmula que puede ser verdadera o falsa. $P \Longrightarrow Q$ es una regla de inferencia.

Conjunción

Las reglas de la conjunción

Reglas de deducción usadas

```
conjI:      ⟦P; Q⟧ ⟹ P ∧ Q
conjunct1: P ∧ Q ⟹ P
conjunct2: P ∧ Q ⟹ Q
```

- **conjI** *introduce* la conjunción: de P y Q, se concluye $P \wedge Q$.
- **conjunct1** y **conjunct2** la *eliminan*: de $P \wedge Q$ se deduce cada parte.

Toda regla de deducción natural viene en pareja: una de introducción y una de eliminación por conectivo.

Ejemplo 1 — p ∧ q, r ⊢ q ∧ r

```
lemma ejemplo_1_1:
  assumes 1: "p ∧ q" and
          2: "r"
  shows "q ∧ r"
proof -
  have 3: "q"
    using 1 by (rule conjunct2)
  show 4: "q ∧ r"
    using 3 2 by (rule conjI)
qed
```

Ejemplo 1 — p ∧ q, r ⊢ q ∧ r

```
lemma ejemplo_1_1:
  assumes 1: "p ∧ q" and
          2: "r"
  shows "q ∧ r"
proof -
  have 3: "q"
    using 1 by (rule conjunct2)
  show 4: "q ∧ r"
    using 3 2 by (rule conjI)
qed
```

Reglas de deducción usadas

```
conjunct2: P ∧ Q ⟹ Q
conjI: ⟦P; Q⟧ ⟹ P ∧ Q
```

Ejemplo 1 — p ∧ q, r ⊢ q ∧ r

```
lemma ejemplo_1_1:
  assumes 1: "p ∧ q" and
          2: "r"
  shows "q ∧ r"
proof -
  have 3: "q"
    using 1 by (rule conjunct2)
  show 4: "q ∧ r"
    using 3 2 by (rule conjI)
qed
```

Reglas de deducción usadas

```
conjunct2: P ∧ Q ⟹ Q
conjI: ⟦P; Q⟧ ⟹ P ∧ Q
```

- **Paso 3:** de la hipótesis 1 (p ∧ q) se deduce q.

Ejemplo 1 — p ∧ q, r ⊢ q ∧ r

```
lemma ejemplo_1_1:
  assumes 1: "p ∧ q" and
          2: "r"
  shows "q ∧ r"
proof -
  have 3: "q"
    using 1 by (rule conjunct2)
  show 4: "q ∧ r"
    using 3 2 by (rule conjI)
qed
```

Reglas de deducción usadas

```
conjunct2: P ∧ Q ⟹ Q
conjI: ⟦P; Q⟧ ⟹ P ∧ Q
```

- **Paso 3:** de la hipótesis 1 (p ∧ q) se deduce q.
- **Paso 4:** junto q (paso 3) con r (hipótesis 2).

La misma prueba, más corta

Dejando la regla implícita con `..` — Isabelle la encuentra:

```
proof -
  have "q" using assms(1) ..
  thus "q ∧ r" using assms(2) ..
qed
```

Automática:

```
lemma ejemplo_1_5: "⟦p ∧ q; r⟧ ⟹ q ∧ r"
by auto
```

Siete líneas $\longrightarrow$ dos.

Doble negación

Ejemplo 2 — p, ¬¬(q ∧ r) ⊢ ¬¬p ∧ r

```
lemma ejemplo_2_1:
  assumes 1: "p" and
          2: "¬¬(q ∧ r)"
  shows "¬¬ p ∧ r"
proof -
  have 3: "¬¬ p"
    using 1 by (rule notnotI)
  have 4: "q ∧ r"
    using 2 by (rule notnotD)
  have 5: "r"
    using 4 by (rule conjunct2)
  show 6: "¬¬ p ∧ r"
    using 3 5 by (rule conjI)
qed
```

Ejemplo 2 — p, ¬¬(q ∧ r) ⊢ ¬¬p ∧ r

```
lemma ejemplo_2_1:
  assumes 1: "p" and
          2: "¬¬(q ∧ r)"
  shows "¬¬ p ∧ r"
proof -
  have 3: "¬¬ p"
    using 1 by (rule notnotI)
  have 4: "q ∧ r"
    using 2 by (rule notnotD)
  have 5: "r"
    using 4 by (rule conjunct2)
  show 6: "¬¬ p ∧ r"
    using 3 5 by (rule conjI)
qed
```

Reglas de deducción usadas

```
notnotI: P ⟹ ¬¬ P
notnotD: ¬¬ P ⟹ P
```

Ejemplo 2 — p, ¬¬(q ∧ r) ⊢ ¬¬p ∧ r

```
lemma ejemplo_2_1:
  assumes 1: "p" and
          2: "¬¬(q ∧ r)"
  shows "¬¬ p ∧ r"
proof -
  have 3: "¬¬ p"
    using 1 by (rule notnotI)
  have 4: "q ∧ r"
    using 2 by (rule notnotD)
  have 5: "r"
    using 4 by (rule conjunct2)
  show 6: "¬¬ p ∧ r"
    using 3 5 by (rule conjI)
qed
```

Reglas de deducción usadas

```
notnotI: P ⟹ ¬¬ P
notnotD: ¬¬ P ⟹ P
```

- Cuatro pasos, cada uno con su regla. Se lee como el razonamiento que uno haría en papel.

Condicional

Ejemplo 3 — el modus ponens

Demostrar $\neg p \wedge q,\ \neg p \wedge q \longrightarrow r \vee \neg p \vdash r \vee \neg p$

```
lemma ejemplo_3_1:
  assumes 1: "¬ p ∧ q" and
          2: "¬ p ∧ q ⟶ r ∨ ¬ p"
  shows      "r ∨ ¬ p"
proof -
  show "r ∨ ¬ p" using 2 1 by (rule mp)
qed
```

Reglas de deducción usadas

```
mp: ⟦P ⟶ Q; P⟧ ⟹ Q
```

Un solo paso. El orden de `using 2 1` importa: primero el condicional, después el antecedente.

Ejemplo 8 — introducir un condicional

Demostrar $p \longrightarrow q \vdash \neg q \longrightarrow \neg p$

```
lemma ejemplo_8_1:
  assumes 1: "p ⟶ q"
  shows "¬ q ⟶ ¬ p"
proof -
  { assume 2: "¬ q"
    have "¬ p" using 1 2 by (rule mt) }
  thus "¬ q ⟶ ¬ p" by (rule impI)
qed
```

Ejemplo 8 — introducir un condicional

Demostrar $p \longrightarrow q \vdash \neg q \longrightarrow \neg p$

```
lemma ejemplo_8_1:
  assumes 1: "p ⟶ q"
  shows "¬ q ⟶ ¬ p"
proof -
  { assume 2: "¬ q"
    have "¬ p" using 1 2 by (rule mt) }
  thus "¬ q ⟶ ¬ p" by (rule impI)
qed
```

Ejemplo 8 — introducir un condicional

Demostrar $p \longrightarrow q \vdash \neg q \longrightarrow \neg p$

```
lemma ejemplo_8_1:
  assumes 1: "p ⟶ q"
  shows "¬ q ⟶ ¬ p"
proof -
  { assume 2: "¬ q"
    have "¬ p" using 1 2 by (rule mt) }
  thus "¬ q ⟶ ¬ p" by (rule impI)
qed
```

Reglas de deducción usadas

```
impI: (P ⟹ Q) ⟹ P ⟶ Q
mt: ⟦P ⟶ Q; ¬ Q⟧ ⟹ ¬ P
```

Ejemplo 8 — introducir un condicional

Demostrar $p \longrightarrow q \vdash \neg q \longrightarrow \neg p$

```
lemma ejemplo_8_1:
  assumes 1: "p ⟶ q"
  shows "¬ q ⟶ ¬ p"
proof -
  { assume 2: "¬ q"
    have "¬ p" using 1 2 by (rule mt) }
  thus "¬ q ⟶ ¬ p" by (rule impI)
qed
```

Reglas de deducción usadas

```
impI: (P ⟹ Q) ⟹ P ⟶ Q
mt: ⟦P ⟶ Q; ¬ Q⟧ ⟹ ¬ P
```

- Las llaves `{ ... }` abren un bloque: dentro vale la suposición ¬q.

Ejemplo 8 — introducir un condicional

Demostrar $p \longrightarrow q \vdash \neg q \longrightarrow \neg p$

```
lemma ejemplo_8_1:
  assumes 1: "p ⟶ q"
  shows "¬ q ⟶ ¬ p"
proof -
  { assume 2: "¬ q"
    have "¬ p" using 1 2 by (rule mt) }
  thus "¬ q ⟶ ¬ p" by (rule impI)
qed
```

Reglas de deducción usadas

```
impI: (P ⟹ Q) ⟹ P ⟶ Q
mt: ⟦P ⟶ Q; ¬ Q⟧ ⟹ ¬ P
```

- Las llaves `{ ... }` abren un bloque: dentro vale la suposición ¬q.
- **impI** convierte ese bloque —«suponiendo ¬q se obtiene ¬p»— en el condicional.

Ejemplo 8 — introducir un condicional

Demostrar $p \longrightarrow q \vdash \neg q \longrightarrow \neg p$

```
lemma ejemplo_8_1:
  assumes 1: "p ⟶ q"
  shows "¬ q ⟶ ¬ p"
proof -
  { assume 2: "¬ q"
    have "¬ p" using 1 2 by (rule mt) }
  thus "¬ q ⟶ ¬ p" by (rule impI)
qed
```

Reglas de deducción usadas

```
impI: (P ⟹ Q) ⟹ P ⟶ Q
mt: ⟦P ⟶ Q; ¬ Q⟧ ⟹ ¬ P
```

- Las llaves { ... } abren un bloque: dentro vale la suposición ¬q.
- **impI** convierte ese bloque —«suponiendo ¬q se obtiene ¬p»— en el condicional.

Dentro del bloque razonamos con $\Longrightarrow$; el resultado es una fórmula con $\longrightarrow$.

Disyunción

Las reglas de la disyunción

Reglas de deducción usadas

```
disjI1: P ⟹ P ∨ Q
disjI2: Q ⟹ P ∨ Q
disjE:  ⟦P ∨ Q; P ⟹ R; Q ⟹ R⟧ ⟹ R
```

- Introducir es fácil: basta una de las dos partes.
- Eliminar exige razonar por casos: hay que llegar a la misma conclusión R suponiendo P y suponiendo Q.

disjE *es la primera regla del tema con tres premisas. Se reconoce el patrón: «o pasa esto o pasa aquello, y en ambos casos. . . ».*

Ejemplo 12 — p ∨ q ⊢ q ∨ p

```
lemma ejemplo_12_1:
  assumes "p ∨ q"
  shows "q ∨ p"
proof -
  have "p ∨ q" using assms by this
  moreover
  { assume 2: "p"
    have "q ∨ p" using 2 by (rule disjI2) }
  moreover
  { assume 3: "q"
    have "q ∨ p" using 3 by (rule disjI1) }
  ultimately show "q ∨ p" by (rule disjE)
qed
```

Ejemplo 12 — p ∨ q ⊢ q ∨ p

```
lemma ejemplo_12_1:
  assumes "p ∨ q"
  shows "q ∨ p"
proof -
  have "p ∨ q" using assms by this
  moreover
  { assume 2: "p"
    have "q ∨ p" using 2 by (rule disjI2) }
  moreover
  { assume 3: "q"
    have "q ∨ p" using 3 by (rule disjI1) }
  ultimately show "q ∨ p" by (rule disjE)
qed
```

- `moreover` va acumulando hechos; `ultimately` se los entrega todos juntos a la regla.

Ejemplo 12 — p ∨ q ⊢ q ∨ p

```
lemma ejemplo_12_1:
  assumes "p ∨ q"
  shows "q ∨ p"
proof -
  have "p ∨ q" using assms by this
  moreover
  { assume 2: "p"
    have "q ∨ p" using 2 by (rule disjI2) }
  moreover
  { assume 3: "q"
    have "q ∨ p" using 3 by (rule disjI1) }
  ultimately show "q ∨ p" by (rule disjE)
qed
```

- `moreover` va acumulando hechos; `ultimately` se los entrega todos juntos a la regla.
- Los tres hechos acumulados son exactamente las tres premisas que pide **disjE**:

Ejemplo 12 — p ∨ q ⊢ q ∨ p

```
lemma ejemplo_12_1:
  assumes "p ∨ q"
  shows "q ∨ p"
proof -
  have "p ∨ q" using assms by this
  moreover
  { assume 2: "p"
    have "q ∨ p" using 2 by (rule disjI2) }
  moreover
  { assume 3: "q"
    have "q ∨ p" using 3 by (rule disjI1) }
  ultimately show "q ∨ p" by (rule disjE)
qed
```

- `moreover` va acumulando hechos; `ultimately` se los entrega todos juntos a la regla.
- Los tres hechos acumulados son exactamente las tres premisas que pide **disjE**:
  1. p ∨ q
  2. de p sale q ∨ p
  3. de q sale q ∨ p

Negación

Las reglas de la negación

Reglas de deducción usadas

```
notI:   (P ⟹ False) ⟹ ¬ P
notE:   ⟦¬ P; P⟧ ⟹ R
FalseE: False ⟹ P
```

- **notI**: para probar ¬P, suponga P y llegue a contradicción.
- **notE**: de P y $\neg P$ sale cualquier cosa.

FalseE *de falso se deduce cualquier fórmula.*

Ejemplo 15 — $\neg p \vee q \vdash p \longrightarrow q$

```
lemma ejemplo_15_1:
  assumes 1: "¬ p ∨ q"
  shows "p ⟶ q"
proof (rule impI)
  assume 2: "p"
  note 1
  thus "q"
  proof (rule disjE)
    { assume 3: "¬ p"
      show "q" using 3 2 by (rule notE) }
  next
    { assume 4: "q"
      show "q" using 4 by this }
  qed
qed
```

Ejemplo 15 — $\neg p \vee q \vdash p \longrightarrow q$

```
lemma ejemplo_15_1:
  assumes 1: "¬ p ∨ q"
  shows "p ⟶ q"
proof (rule impI)
  assume 2: "p"
  note 1
  thus "q"
  proof (rule disjE)
    { assume 3: "¬ p"
      show "q" using 3 2 by (rule notE) }
  next
    { assume 4: "q"
      show "q" using 4 by this }
  qed
qed
```

- **Hacia atrás:** `proof (rule impI)` dice con qué regla se cerrará antes de probar nada.

Ejemplo 15 — $\neg p \vee q \vdash p \longrightarrow q$

```
lemma ejemplo_15_1:
  assumes 1: "¬ p ∨ q"
  shows "p ⟶ q"
proof (rule impI)
  assume 2: "p"
  note 1
  thus "q"
  proof (rule disjE)
    { assume 3: "¬ p"
      show "q" using 3 2 by (rule notE) }
  next
    { assume 4: "q"
      show "q" using 4 by this }
  qed
qed
```

- **Hacia atrás:** `proof (rule impI)` dice con qué regla se cerrará antes de probar nada.
- Dentro, **disjE** abre los dos casos, separados por `next`.

Ejemplo 15 — $\neg p \vee q \vdash p \longrightarrow q$

```
lemma ejemplo_15_1:
  assumes 1: "¬ p ∨ q"
  shows "p ⟶ q"
proof (rule impI)
  assume 2: "p"
  note 1
  thus "q"
  proof (rule disjE)
    { assume 3: "¬ p"
      show "q" using 3 2 by (rule notE) }
  next
    { assume 4: "q"
      show "q" using 4 by this }
  qed
qed
```

- **Hacia atrás:** `proof (rule impI)` dice con qué regla se cerrará antes de probar nada.
- Dentro, **disjE** abre los dos casos, separados por `next`.
- **Caso** $\neg p$: junto con p da contradicción, y **notE** produce la q.

Ejemplo 15 — $\neg p \vee q \vdash p \longrightarrow q$

```
lemma ejemplo_15_1:
  assumes 1: "¬ p ∨ q"
  shows "p ⟶ q"
proof (rule impI)
  assume 2: "p"
  note 1
  thus "q"
  proof (rule disjE)
    { assume 3: "¬ p"
      show "q" using 3 2 by (rule notE) }
  next
    { assume 4: "q"
      show "q" using 4 by this }
  qed
qed
```

- **Hacia atrás:** `proof (rule impI)` dice con qué regla se cerrará antes de probar nada.
- Dentro, **disjE** abre los dos casos, separados por `next`.
- **Caso** $\neg p$: junto con p da contradicción, y **notE** produce la q.
- **Caso** q: ya la tenemos.

Bicondicional y reglas clásicas

Bicondicional

Reglas de deducción usadas

```
iffI:  ⟦P ⟹ Q; Q ⟹ P⟧ ⟹ P ⟷ Q
iffD1: ⟦Q ⟷ P; Q⟧ ⟹ P
iffD2: ⟦P ⟷ Q; Q⟧ ⟹ P
```

Probar una equivalencia es probar las dos implicaciones.

Ejemplo 24 — razonar por contradicción

Demostrar $\neg p,\ p \vee q \vdash q$

```
lemma ejemplo_24_1:
  assumes "¬ p"
          "p ∨ q"
  shows "q"
using ‘p ∨ q‘
proof (rule disjE)
  assume "p"
  with assms(1) show "q" by contradiction
next
  assume "q"
  thus "q" by assumption
qed
```

Ejemplo 24 — razonar por contradicción

Demostrar $\neg p,\ p \vee q \vdash q$

```
lemma ejemplo_24_1:
  assumes "¬ p"
          "p ∨ q"
  shows "q"
using ‘p ∨ q‘
proof (rule disjE)
  assume "p"
  with assms(1) show "q" by contradiction
next
  assume "q"
  thus "q" by assumption
qed
```

Ejemplo 24 — razonar por contradicción

Demostrar $\neg p,\ p \vee q \vdash q$

```
lemma ejemplo_24_1:
  assumes "¬ p"
          "p ∨ q"
  shows "q"
using ‘p ∨ q‘
proof (rule disjE)
  assume "p"
  with assms(1) show "q" by contradiction
next
  assume "q"
  thus "q" by assumption
qed
```

Reglas de deducción usadas

```
ccontr:
  (¬ P ⟹ False) ⟹ P
```

Ejemplo 24 — razonar por contradicción

Demostrar $\neg p,\ p \vee q \vdash q$

```
lemma ejemplo_24_1:
  assumes "¬ p"
          "p ∨ q"
  shows "q"
using ‘p ∨ q‘
proof (rule disjE)
  assume "p"
  with assms(1) show "q" by contradiction
next
  assume "q"
  thus "q" by assumption
qed
```

Reglas de deducción usadas

```
ccontr:
  (¬ P ⟹ False) ⟹ P
```

- by contradiction cierra el caso: tenemos p y ¬p a la vez.

Ejemplo 24 — razonar por contradicción

Demostrar $\neg p,\ p \vee q \vdash q$

```
lemma ejemplo_24_1:
  assumes "¬ p"
          "p ∨ q"
  shows "q"
using ‘p ∨ q‘
proof (rule disjE)
  assume "p"
  with assms(1) show "q" by contradiction
next
  assume "q"
  thus "q" by assumption
qed
```

Reglas de deducción usadas

```
ccontr:
  (¬ P ⟹ False) ⟹ P
```

- by contradiction cierra el caso: tenemos p y ¬p a la vez.
- **ccontr** es clásica: no vale en lógica intuicionista. Es la que permite «supongamos que no. . . ».

Resumen

Todas las reglas del tema

Conectivo	Introducción	Eliminación
$\wedge$	**conjI**	**conjunct1**, **conjunct2**, **conjE**
$\vee$	**disjI1**, **disjI2**	**disjE**
$\longrightarrow$	**impI**	**mp**, **impE**
$\neg$	**notI**	**notE**
$\longleftrightarrow$	**iffI**	**iffD1**, **iffD2**
$\bot$	—	**FalseE**

Derivadas: **mt**, **notnotI**, **notnotD** **Clásicas:** **ccontr**, **classical**, **excluded_middle**

- Cada paso de una prueba dice de qué hechos se deduce y con qué reglas.
- Cada conectivo trae su pareja de reglas: introducción y eliminación.
- Distinga $\longrightarrow$ (fórmula) de $\Longrightarrow$ (regla). Es el error más común del tema.
- Escriba primero la versión detallada. La automática viene después, y se entiende porque ya sabe qué está automatizando.